

Apache Hive Introduction:

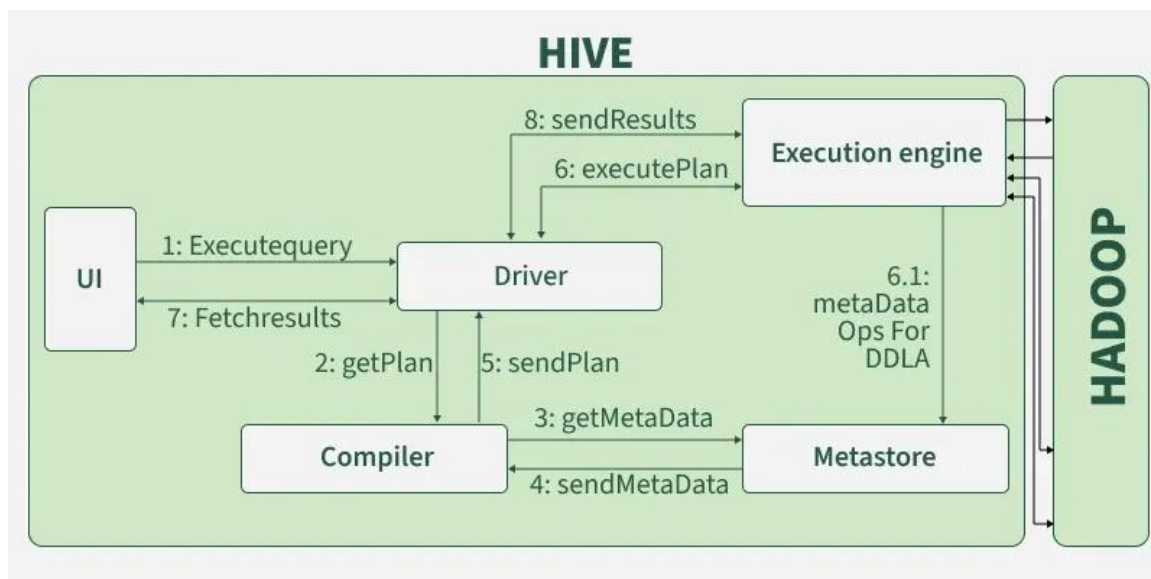
- 1.Hive is a data warehouse infrastructure tool to process structured data in Hadoop.
- 2.It resides on top of Hadoop to summarize Big Data, and makes querying and analyzing easy.
- 3.Initially Hive was developed by Facebook, later the Apache Software Foundation took it up and developed it further as an open source under the name Apache Hive.
- 4.It is used by different companies. For example, Amazon uses it in Amazon Elastic MapReduce.

Features of Apache Hive:

- 1.It stores schema in a database and processed data into HDFS.
- 2.It is designed for Online Analytical Processing (OLAP).
- 3.It provides SQL type language for querying called HiveQL or HQL.
- 4.It is familiar, fast, scalable, and extensible.

Hive is not:

- 1.A relational database.
- 2.A design for Online Transaction Processing (OLTP).
- 3.A language for real-time queries and row-level updates.

Hive Architecture:

1.UI: Tools like CLI (Command Line Interface), Hive Web UI, JDBC, and ODBC are provided for users to submit HiveQL queries. Accepts queries and passes them to the Driver for processing.

2.Driver: The Driver receives HiveQL queries from the user interface.

It manages the entire query lifecycle, including parsing, compiling, executing and returning results.

3.Compiler: The Compiler checks the syntax of the query and converts HiveQL into an execution plan. It also interacts with the Metastore to understand table schemas and data locations.

4.Metastore: The Metastore stores metadata information such as: Table names, Column names and data types, HDFS locations, Partitions. It acts like a catalog for Hive tables. Typically backed by an RDBMS (e.g., MySQL, PostgreSQL).

5.Execution Engine: It is connected to the Hadoop Framework. The Execution Engine executes the query plan generated by the compiler. It converts the plan into MapReduce, Tez or Spark jobs and runs them on Hadoop.

6.Hadoop Storage (HDFS): Actual data is stored in HDFS or cloud storage like S3. Hive does not store data itself; it only queries data from Hadoop storage.

Metastore:

1.The Metastore is a central component of Apache Hive that stores metadata information about the structure and location of data.

2.It does not store the actual data; instead, it maintains information such as database names, table definitions, column names and data types, partitions, and the storage location of data in HDFS or cloud storage.

3.The Metastore allows Hive to understand how data is organized so that queries can be correctly parsed and executed.

4.During query compilation, Hive consults the Metastore to retrieve schema and location details, which helps in generating an efficient execution plan.

5.What info does the Hive metastore stores?

It stores db names and details, Table names and table types (managed or external), Column names, data types and column order, Storage location of data in HDFS or cloud storage, File format information, Table properties and configuration settings, etc.

Why Hive Metastore is required:

1.It helps Hive understand the structure and schema of data stored in HDFS or cloud storage.

2.During query execution, Hive uses the Metastore to locate data files and read them correctly.

3.It allows multiple users and tools to share the same metadata information.

4.Without the Metastore, Hive would not know where data is stored or how it is organized.

5.Hive Metastore is required to store and manage metadata about databases, tables and columns.

Hive Table:

1.A Hive table is a logical structure in Apache Hive used to organize and represent data stored in Hadoop.

2.Hive tables do not store data themselves; instead, they define the schema and location of data stored in HDFS or cloud storage.

3.Types of Tables in Hive:

i. Managed Table (Internal Table):

- Hive manages both data and metadata.
- Data is stored in Hive's default warehouse directory.
- When the table is dropped, both the table and its data are deleted.
- Used when Hive has full control over the data.

ii. External Table:

- Hive manages only the metadata, not the actual data.
- Data is stored at an external location (HDFS or cloud storage).
- When the table is dropped, only metadata is removed; data remains safe.
- Used when data is shared with other tools or systems.

Basic query of Hive:

1. Create Database: **CREATE DATABASE studentdb;**
2. Use the Database: **USE studentdb;**
3. Create Table: **CREATE TABLE student (id INT, name STRING, marks INT);**
4. Insert Values into Table: **INSERT INTO student VALUES (1, 'Amit', 85), (2, 'Riya', 90), (3, 'Karan', 78);**
5. Show Tables: **SHOW TABLES;**
6. Select Data from Table: **SELECT * FROM student;**
7. Drop the Table: **DROP TABLE student;**

Joins: A JOIN in Hive is used to combine data from two or more tables based on a common column. It is mainly used to retrieve related data stored in different tables.

Syntax:

SELECT columns FROM table1 JOIN table2 ON table1.common_column = table2.common_column;

Ex:

SELECT s.id, s.name, m.marks FROM student s JOIN marks m ON s.id = m.id;

Types of Joins:

1. **Inner Join:** Returns only matching records from both tables.

Ex:

SELECT s.id, s.name, m.marks FROM student s INNER JOIN marks m ON s.id = m.id;

OUTPUT:

id	name	marks
1	Amit	85
2	Riya	90

2. **Left Outer Join:** Returns all records from the left table and matching records from the right table.

Ex:

SELECT s.id, s.name, m.marks FROM student s LEFT OUTER JOIN marks m ON s.id = m.id;

OUTPUT:

id	name	marks
1	Amit	85
2	Riya	90
3	Karan	NULL

3. **Right Outer Join:** Returns all records from the right table and matching records from the left table.

Ex:

SELECT s.id, s.name, m.marks FROM student s RIGHT OUTER JOIN marks m ON s.id = m.id;

OUTPUT:

id	name	marks
1	Amit	85
2	Riya	90
4	NULL	75

4.Full Outer Join: Returns all records from both tables.

Ex:

```
SELECT s.id, s.name, m.marks FROM student s FULL OUTER JOIN marks m ON s.id = m.id;
```

OUTPUT:

id	name	marks
1	Amit	85
2	Riya	90
3	Karan	NULL
4	NULL	75

5.Left Semi Join: A Left Semi Join in Hive is used to return only the rows from the left table that have matching values in the right table. It does not return columns from the right table. It only checks whether a matching record exists in the right table.

Syntax:

```
SELECT * FROM table1 LEFT SEMI JOIN table2 ON table1.column = table2.column;
```

Ex:

```
SELECT s.id, s.name FROM student s LEFT SEMI JOIN marks m ON s.id = m.id;
```

OUTPUT:

id	name
1	Amit
2	Riya

Hive Index:

- 1.Hive indexing is a technique used to improve query performance by reducing the amount of data scanned during execution.
- 2.When an index is created on a column, Hive generates a separate index table that stores indexed column values along with references to the corresponding data blocks in HDFS.
- 3.During query execution, Hive checks whether a relevant index exists and, if applicable, uses the index table to locate only the required data blocks instead of scanning the entire table.
- 4.Indexes in Hive must be manually rebuilt after data insertion or modification because they are not automatically maintained in real time.
- 5.Although indexing was introduced to optimize performance, it has been deprecated in newer versions of Hive due to maintenance complexity and limited efficiency in distributed environments.

Hive Bucketing:

- 1.Bucketing in Hive is a technique used to organize data into fixed number of buckets (files).
- 2.It distributes data based on the hash value of a column, so rows with the same column value go into the same bucket.
- 3.Bucketing is useful when partitioning alone is not sufficient or when further data organization is required.
- 4.Each bucket is stored as a separate file in HDFS inside the table or partition directory.
- 5.Bucketing improves query performance, especially during joins and sampling.

Syntax:

```
CREATE TABLE table_name (column1 datatype, column2 datatype) CLUSTERED BY  
(column_name) INTO number_of_buckets BUCKETS;
```

Ex:

```
CREATE TABLE student (  
  id INT,  
  name STRING,  
  marks INT  
)  
CLUSTERED BY (id)  
INTO 4 BUCKETS;
```

Group by clause in Hive: The GROUP BY clause in Hive is used to group rows that have the same values in a particular column. It is mainly used with aggregate functions like COUNT, SUM, AVG, MAX and MIN to perform calculations on each group.

Syntax:

```
SELECT column_name, aggregate_function(column_name) FROM table_name GROUP BY  
column_name;
```

Ex:

```
SELECT class, COUNT(*) FROM student GROUP BY class;
```

Partitioning in Hive: Partitioning in Hive is a technique used to divide a large table into smaller parts based on the value of a column. Each partition is stored as a separate folder in HDFS. It improves query performance because Hive reads only the required partition instead of the entire table.

1.Static Partitioning: In static partitioning, the partition value is specified manually while inserting data.

- User provides partition value in the query.
- Suitable when partition values are known in advance.

Syntax:

```
INSERT INTO TABLE table_name PARTITION (column='value') SELECT * FROM source_table;
```

2.Dynamic Partitioning: In dynamic partitioning, the partition value is automatically created based on the data being inserted.

- Hive determines partition values at runtime.
- Suitable when partition values are not known beforehand.

Syntax:

```
INSERT INTO TABLE table_name PARTITION (column) SELECT column1, column2 FROM  
source_table;
```

MapReduce using Hive: When a Hive query is executed, Hive converts it into **MapReduce jobs** for processing data in Hadoop.

1.Map Phase: In the Map phase, input data from HDFS is divided into blocks and processed in parallel by mapper tasks.

Each mapper reads its portion of data and converts it into **key-value pairs** based on the query condition.

2.Shuffle and Sort Phase: In this phase, all intermediate key-value pairs produced by mappers are **grouped and sorted by key**.

Data with the same key is sent to the same reducer. This step ensures proper grouping before final processing.

3.Reducer Phase: In the Reducer phase, grouped data is processed to produce the final result. Operations such as **GROUP BY, COUNT, SUM, JOIN** are completed here.

The final output is then written back to HDFS.

Analysis of MapReduce Execution: Analysis of MapReduce execution in Hive refers to the process by which a Hive query is internally converted into one or more MapReduce jobs and executed on the Hadoop cluster.

Phases Involved in Hive Query Execution

1.Parsing Phase: Hive checks the syntax of the query and generates a parse tree. If any syntax error is found, the query execution stops.

2.Semantic Analysis Phase: Hive verifies the tables, columns and data types using metadata from the Metastore. It ensures the query is logically valid.

3.Logical Plan Generation: Hive creates a logical execution plan that defines operations such as filtering, grouping, joining and sorting.

4.Physical / MapReduce Plan Generation: The logical plan is converted into a physical plan. Hive determines how many MapReduce jobs are required and how tasks will be distributed.

5.MapReduce Job Breakdown: Once Hive has generated the plan, MapReduce executes it

1.Map Phase: Reads input data and produces intermediate key-value pairs.

2.Shuffle and Sort Phase: Groups and sorts intermediate data.

3.Reduce Phase: Performs final computations and writes output to HDFS.

Hive Optimization: Hive optimization refers to techniques used to improve query performance and reduce execution time while processing large datasets in Hadoop. Optimization mainly focuses on reducing data, scanning and data movements.

Important Optimization Techniques:

1.Partitioning: Partitioning divides a large Hive table into smaller parts based on the value of a column, such as date or region. Each partition is stored as a separate directory in HDFS. During query execution, Hive scans only the required partitions instead of the entire table, which significantly reduces data scanning and improves performance.

2.Bucketing: Bucketing divides data into a fixed number of buckets using a hash function on a column. Unlike partitioning, the number of buckets is predefined. Bucketing improves query performance, especially in joins and sampling, because related records are stored in the same bucket.

3.Use of ORC File Format: ORC (Optimized Row Columnar) is a columnar storage format that supports compression and indexing. It stores data in a compact and efficient manner. Since

Hive reads only required columns instead of entire rows, ORC significantly improves query speed and reduces storage space.

4.Using WHERE Clause and Avoiding SELECT: Using a WHERE clause helps filter data early in the execution process, reducing the amount of data processed. Avoiding SELECT * ensures that only required columns are read from the table, which minimizes I/O operations and improves query performance.

5.Statistics and Cost-Based Optimization (CBO): Hive can collect table and column statistics such as number of rows and data distribution. Cost-Based Optimization uses these statistics to select the most efficient execution plan, choose the best join order and reduce unnecessary processing.

Implementation of MapReduce program with HiveQL: Hive allows users to implement MapReduce programs by writing HiveQL queries instead of writing Java MapReduce code. When a HiveQL query is executed, Hive automatically converts it into one or more MapReduce jobs and runs them on Hadoop.

Hive implements MapReduce internally when a HiveQL query is executed. The process involves the following stages:

1.Parsing and Compilation: In this stage, Hive checks the syntax of the HiveQL query and verifies tables, columns and data types using the Metastore. After validation, Hive creates a logical execution plan for the query.

2.Conversion to MapReduce Jobs: The logical plan is converted into one or more MapReduce jobs. Hive determines how many MapReduce stages are required based on operations like JOIN, GROUP BY or ORDER BY.

3.Map Phase: In the Map phase, input data from HDFS is read and processed. The mapper generates intermediate key-value pairs according to the query logic.

4.Shuffle and Sort Phase: The intermediate key-value pairs are grouped and sorted by key. Data with the same key is sent to the same reducer for further processing.

5.Reduce Phase: In the Reduce phase, grouped data is processed to perform final operations such as aggregation or joining. The final output is written back to HDFS.